

# Resumen Técnico: Contenedor Docker - Microservicio de Agendamiento

Line spacing: 1.25

## 1. Concepto y Arquitectura

El microservicio de Agendamiento está empaquetado en un contenedor Docker basado en Python 3.12-slim, encapsulando la API REST (Flask), sus dependencias y configuración. Esto garantiza un comportamiento idéntico en cualquier entorno de desarrollo o servidor.

La arquitectura completa se gestiona mediante Docker Compose, que orquesta dos servicios en una red privada interna (red-telemedicina):

- **api:** Contenedor local (Flask) en el puerto 5000. Ejecuta las migraciones antes de arrancar.
- **db-postgres:** Base de datos PostgreSQL 15-alpine en el puerto 5432, con persistencia mediante el volumen datos\_postgres.

## 2. Construcción del Contenedor (Dockerfile Keypoints)

La construcción de la imagen optimiza el uso de la caché de capas y la seguridad mediante los siguientes pasos clave:

1. **Imagen Base:** Python 3.12-slim para reducir el tamaño y mejorar la seguridad.
2. **Entorno:** Configuración de variables para evitar archivos .pyc y forzar logs en tiempo real.
3. **Dependencias:** Copia aislada de requirements.txt e instalación limpia (instalando herramientas de compilación como gcc y eliminándolas posteriormente).
4. **Código Fuente:** Copia posterior del código para maximizar la caché de capas.
5. **Producción:** Configuración de Flask en modo producción (DEBUG=False, cookies seguras, CORS restringido).

## 3. Componentes y Tecnologías

| Categoría         | Tecnologías / Componentes clave                                                                                                                                                      |
|-------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Dependencias Core | Flask (3.0.3), SQLAlchemy + Flask-Migrate (2.0.30) para ORM y migraciones, Flask-JWT-Extended (4.6.0) para autenticación, Marshmallow (3.21.3) para validación, psycopg2 y Werkzeug. |
| Código Fuente     | 7 modelos SQLAlchemy, 7 Blueprints de Flask                                                                                                                                          |

| Categoría                   | Tecnologías / Componentes clave                                                                                                    |
|-----------------------------|------------------------------------------------------------------------------------------------------------------------------------|
|                             | (rutas), lógica de negocio (validaciones y máquina de estados), esquemas Marshmallow y configuraciones por entorno.                |
| <b>Variables de Entorno</b> | DATABASE_URL, SECRET_KEY, JWT_SECRET_KEY, duración del token (JWT_ACCESS_TOKEN_EXPIRES), FLASK_ENV y configuración de host/puerto. |

*Autores: Darwin García | Juan José Figueroa Ayala (Plataforma de Telemedicina y Triage, 2026).*